Nailing Release 1.0

while optimizing for speed, spend, and scope.

ASH MAURYA
CONTINUOUS INNOVATION
PROBLEM/SOLUTION FIT

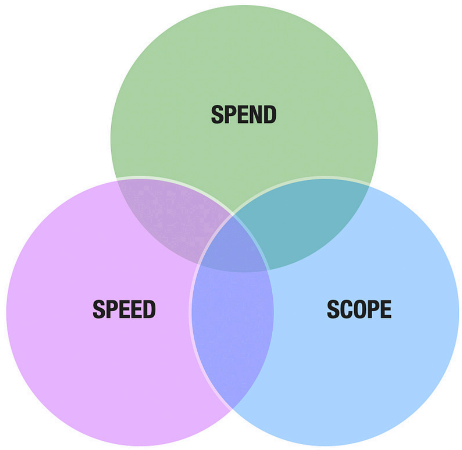
I recently declared that I'm moving away from MVP in favor of:

MDVFP (pronounced MD VFP)

M: Minimum (the smallest feature set)
D: Desirable (that customers want,)
V: Viable (that makes your business model work)
F: Feasible (that you can build quickly)
P: Product (and package as release 1.0)

The MDVFP is the smallest desirable, viable, and feasible solution that causes a switch from an old way (existing alternatives) to your new way, i.e., customers fire the old way to hire your new way.

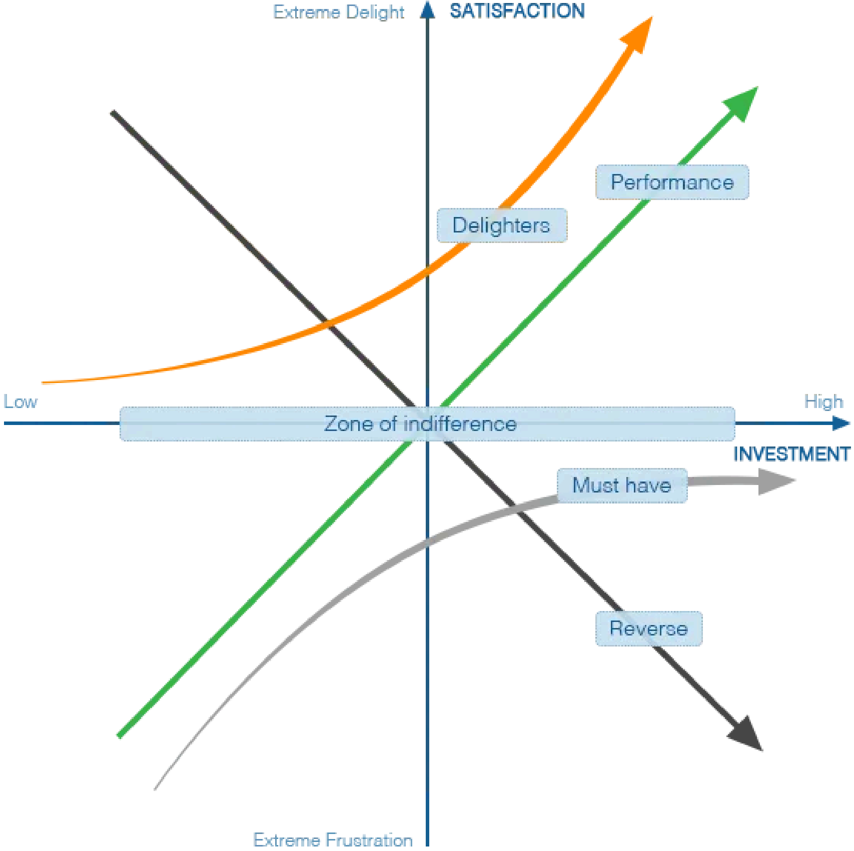
To do this effectively, you have to carefully balance **speed, spend, and scope**.



In this post, I'm going to show you how to embrace these constraints and use the Kano model to help you assemble the right feature cocktail for your Release 1.0.

A Kano Model Primer

Noriaki Kano developed the Kano model in the 1980s, where he identified five types of product features by plotting customer satisfaction against company investment (in a given feature):



It took me more than a cursory glance to understand the chart above. If you're new to the Kano model, this will be time well spent.

5 Types of Product Features

1. **Must have**
   These are basic features that, when absent, make customers unhappy, but their presence doesn't create off-scale happiness. They are expected.

   Example: Car brakes

2. **Zone of indifference**
   These features have no impact on customer satisfaction.

   Example: The color of your car's filter.

3. **Performance**
   These features have a positive linear

relationship with customer satisfaction, i,.e., more is better.

Example: Car horsepower.

4. **Delighters**
These features create off-scale happiness when present, but their absence doesn’t make customers unhappy. They are unexpected.

Example: Park-assist.

5. **Reverse**
These features have a negative linear relationship with customer satisfaction, i.e., more is worse.

Example: Car emissions.

Feature Decay

Kano also found that a feature’s ability to delight has a finite lifetime. Over time, a delighter feature becomes a performance feature and eventually a must-have (basic expectation).

Examples:

- The multi-touch interface first revealed in the iPhone (delighter) is now a must-have on any mobile phone (must-have).
- Park-assist cameras/features are increasingly becoming standard features on cars.

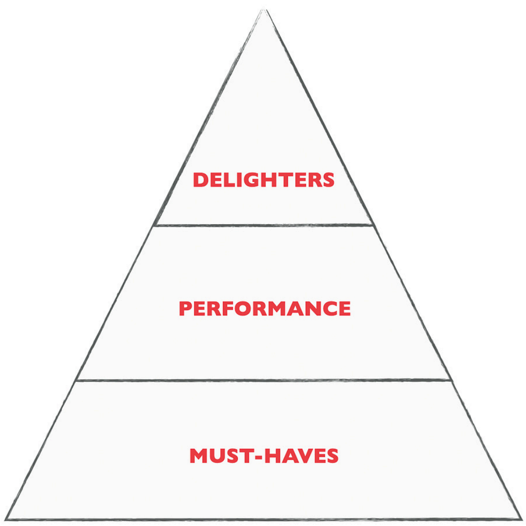
Applying the Kano Model

Given that a startup is constrained with limited resources (speed, spend, and scope), they need to **maximize their product's unique value proposition while minimizing on features**.

It's easy to start with what not to do.

Stay away from the zone of indifference.

We can then theorize that the right feature cocktail needs to be some mix of delighters, performance, and must-have features (in that order of priority).



Here's why.

- Must-have features, while important, have diminishing returns. If you only implement must-have features, the incumbent wins. You don't cause a switch.

- To cause a switch, you need to aim for 3x-10x better. If your performance features can outperform at that level, you stand a chance to cause a switch. Otherwise, the incumbent wins again.
- Aiming for 3-10x better on performance is no easy task. This is why delighters are so crucial for a startup. These features don't exist in the incumbent's offering, so they are unique and create off-scale happiness. That's a textbook definition of a unique value proposition.

Now the bad news,

- Even though delighters seem a no-brainer, they typically require the most innovation (and investment). You can only afford to pack in 1 or 2, so you have to pick carefully.
- Even though must-have features have diminishing returns, they are still required. Can you imagine buying a Tesla without brakes or an iPhone that couldn't make calls? How do you reduce scope?
- Finally, remember that all delighters downgrade at some point to performance features, so these features aren't simply about making the customer happy. They need to be tied to a measurable (performance) metric too.

To see how to design around these challenges, I'll illustrate with a case study: Tesla.

Background: When Tesla announced its first electric car in 2006, it had no experience building cars or prototypes. Yet, they presold and launched their MDVFP product (Telsa Roadster) in just 2.5 years. Other car companies take roughly ten years to go from concept to a road-drivable vehicle.

How did Tesla launch a car four times faster?

1. Build your UVP around a single delighter feature (Gamechanger)

Your MDFVP is all about delivering on your UVP (or desirability), so I recommend starting there.

The UVP of an electric car is sustainability, and when messaged as a reverse feature it equates to emissions.

In 2006 we had varying levels of high to low-emission vehicles (including hybrids). However, when you go to extremes and promise zero emissions and tie it to an emotional fossil-fuel free future story, you turn emissions into a delighter feature.

Happiness is an emotion that follows a logarithmic scale versus a linear one which is why delighters pack such a great punch.

2. Ensure your delighter feature passes a minimum performance metric (Not a showstopper)

Even though delighter features trigger on emotion, they still need to be measurable and comparable to other alternatives to cause a switch. In the case of Tesla, this equated to the range of the car.

A typical car can go at least 200 miles (300 km) on a full tank which became their minimum performance metric to meet to overcome switching interia.

Over time, this was also how they also created pricing tiers.

More range = Higher pricing.

3. Layer on at least one other axis of better (Strengthen Positioning)

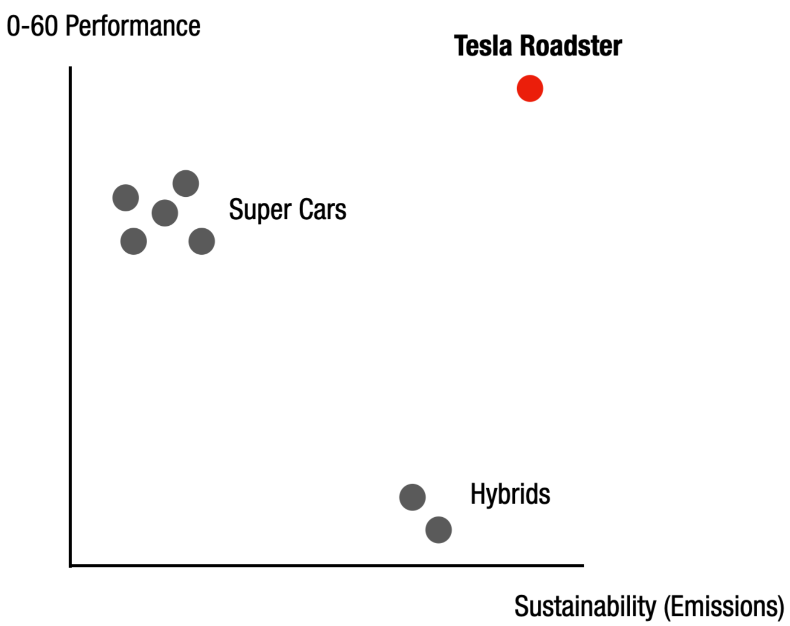
Being better in one dimension may be enough, but two is even better and more defensible. As it's harder to build multiple delighters from scratch, look for other side-effects of your new way that can be positioned as performance or delighter features.

For Tesla, this came from the realization that because electric cars deliver higher torque, they can accelerate faster than other cars from a stop.

Delighter positioning: Electric cars don't have to be slow and boring. They can be supercars.

They ran stunt drag races against other supercars and claimed they had built the *fastest, greenest car in the world*.

This made for a powerful 2x2 differentiation matrix like the one below:



They still use this positioning with their "faster" cars which have a Ludicrous mode and clock a 0-60 time of 2.3 seconds!

Over time, they added another delighter side-effect to their positioning: *Because Teslas don't have an engine block or oil, they are much safer in crashes.*

Delighter positioning: Fastest/safest, greenest car in the world.

All this is great, but how did they build their Release 1.0 (Tesla Roadster) four times faster than other car companies?

Cars have many basic features like brakes, aesthetics, and hundreds of other little features we take for granted. How did an upstart company that had never manufactured a car at any scale at that point manage to package all these must-have features in record time?

The answer:

4. Innovate around basic features (Avoid distractions)

Basic features are required, but that doesn't mean you have to be the one that builds them or that you have to deliver them all the once in Release 1.0.

Tesla devised a brilliant strategy for providing basic car features without building a single one by licensing a prebuilt car from another car company (Lotus Motors) and retrofitting it with their electric battery.

Lotus Elise

Electric Lotus Elise = Tesla Roadster

The car on the left is a sports car. The car on the right is an electric sports car. That's the unique difference that mattered. Delivering on this difference required innovating on a better battery, not a better car.

Cobbling together basic features from other pre-existing components/products is a "Wizard of Oz" recipe for innovating around basic features leaving room to invest more heavily in your delighter feature(s). There are many more such recipes.

In Summary…

Here are the steps to nailing your Release 1.0 for MDVFP while embracing your speed, spend, and scope constraints:

- Build your game-changing UVP around something different that matters to customers (Delighter feature),
- Overcome switching inertia by tying your delighter feature to a performance metric,
- Strengthen your positioning by reframing related benefits as delighters,
- Aim for a 3-10x better story,
- Avoid distractions by innovating around basic features so you can invest more in your delighter and performance features.

Read Next: